Arrow workshop: Resilient delivery quotes

A self-guided introduction for Kotlin developers

Arrow 2.3 · Kotlin 2.4 · November 2026

Contents

Build one thing **1**
- Audience . . . 1
- What you will learn . . . 2
- The finished use case . . . 2
- Error design . . . 2

Setup **2**

Workshop **3**
- 1. Domain and immutable data . . . 3
- 2. Validation: report everything wrong . . . 4
- 3. Typed domain errors . . . 5
- 4. Absence and existing Kotlin APIs . . . 6
- 5. Parallel courier quotes . . . 6
- 6. Live quote updates . . . 8
- 7. Resilient provider integrations . . . 8
- 8. Resource safety . . . 9

Optional extras **9**
- Optics . . . 9
- Racing . . . 10
- Transactional memory . . . 10
- Sagas . . . 10

Testing guide **10**

Further reading **11**

Questions to take away **11**

Build one thing

Build a small, framework-free **Delivery Quote Service**. It accepts a shipment request, validates it, asks courier providers for prices, returns the best quote, and behaves sensibly when providers are slow or fail.

The code uses deterministic fake providers. Do not add HTTP, a database, or dependency injection unless you want to after the workshop. The purpose is to practise Arrow in ordinary Kotlin business code.

Audience

Kotlin developers who are comfortable with object-oriented Kotlin and familiar with collection functions such as `map` and `filter`. You do not need formal functional-programming theory. Some coroutine experience (`suspend`, `launch`, `delay`) helps from section 5 on.

What you will learn

- Model expected failures explicitly.
- Separate invalid input, domain failure, absence, and unexpected exceptions.
- Use the `Raise` DSL with context parameters.
- Use `Either`, `Option`, and `Result` at suitable boundaries.
- Accumulate independent validation failures.
- Run independent suspend functions in parallel, stream results, and race redundant work.
- Release resources on success, typed failure, exception, and cancellation.
- Add retries and circuit breakers to unstable integrations.
- Understand where immutable data and regular nested `copy` become uncomfortable.

The finished use case

The final public boundary is deliberately explicit:

```
class QuoteService(zones: LegacyZoneLookup, providers: List<CourierProvider>) {
    suspend fun quoteShipment(request: ShipmentRequest): Either<QuoteShipmentError,
↪   DeliveryQuote>
}
```

Internally, use direct-looking code with `Raise<QuoteError>`. At the boundary, return `Either` so a caller can see that quoting can fail. This is not a rule for every program. It is a useful contrast between the two Arrow styles. Constructor parameters are all the wiring the workshop needs; there is no dependency-injection framework.

Error design

Use different error types because callers make different decisions for each.

Situation	Representation	Example response
Invalid request fields	Accumulated `ShipmentValidationError` values	Show every field problem to the caller
A valid request cannot be quoted	`QuoteError` raised in `Raise`	Say why no quote is available
One courier will not quote	`CourierDecline` value	Skip that courier, or stop everything if it is fatal
Optional information is absent	Nullable value or `Option`	Continue without insurance or pickup time
A provider throws or times out	Exception at the integration edge, classified before recovery	Retry, open a circuit, or report technical failure

The distinction is the core design exercise. Do not use exceptions for ordinary business outcomes, and do not collect unrelated domain failures merely because validation does.

Setup

Clone the workshop repository. Everything runs through the Gradle wrapper, so you only need **Java 25**. We recommend Eclipse Temurin; if no Java 25 is installed, the Foojay toolchain resolver downloads one.

The repository is a single Gradle build:

Directory	Contents
`starter/`	Empty workspace with all dependencies. Do the exercises here.
`01-domain/ ... 08-resource-safety/`	Reference solution after each section. Each is a complete copy, so `diff -r 04-boundaries/src 05-parallel-quotes/src` shows exactly what section 5 adds.

Directory	Contents
`09-extras/`	Reference solutions for the optional extras.

```
./gradlew :starter:test             # your work
./gradlew :05-parallel-quotes:test  # one reference stage
./gradlew check                     # every stage, plus kotlinter and detekt
```

Versions are pinned in `gradle/libs.versions.toml`: Kotlin 2.4.21, Gradle 9.8.2, Arrow 2.3.0, Kotest 6.2.6, and kotlinx-coroutines 1.11.0. Context parameters are stable in Kotlin 2.4, so no compiler flag is needed. A stage's dependencies look like this:

```
dependencies {
    implementation(platform(libs.arrow.stack))  // the Arrow BOM keeps all modules on one version
    implementation(libs.bundles.arrow)          // arrow-core, arrow-fx-coroutines,
↪   arrow-resilience

    testImplementation(libs.bundles.test)       // Kotest runner and assertions,
↪   kotest-assertions-arrow,
                                                // kotlinx-coroutines-test
}
```

`kotest-assertions-arrow` gives `shouldBeRight()` and `shouldBeLeft()`, which return the contained value. Use `kotlinx-coroutines-test` when testing time, cancellation, flows, retries, or circuit breakers.

Packages used by the reference solution:

```
workshop/
  domain/       data model
  validation/   request validation and the validated Shipment
  quoting/      errors, rules, providers, and the quote service
  providers/    legacy and session-based integrations
  resilience/   retry and circuit breaker adapters
```

Workshop

Work through the sections in order. Sections 1–5 form a complete vertical slice and are the recommended live-session route. Sections 6–8 continue directly from it when you have more time.

1. Domain and immutable data

Start with plain Kotlin data classes. Keep values immutable and use small value types where they add clarity.

```
@JvmInline
value class Money(val cents: Long) : Comparable<Money> {
    init { require(cents >= 0) { "Money cannot be negative: $cents" } }
    operator fun plus(other: Money): Money = Money(cents + other.cents)
    override fun compareTo(other: Money): Int = cents.compareTo(other.cents)
    companion object { val ZERO = Money(0) }
}

enum class DeliveryService(val maxDeliveryDays: Int) { STANDARD(5), EXPRESS(1) }

data class ShipmentRequest(
    val sender: Address,
    val recipient: Address,
    val parcels: List<Parcel>,
    val service: DeliveryService,
    val options: DeliveryOptions = DeliveryOptions(),
)

data class Address(val name: String, val street: String, val postcode: String, val country:
↪   Country)
data class Parcel(val weightGrams: Int, val dimensions: Dimensions)
```

```
data class DeliveryOptions(val insuranceAmount: Money? = null, val pickupWindow: PickupWindow? =
↪  null)
```

Add `Country`, `Courier`, `Dimensions`, `PickupWindow`, `CourierQuote`, and `DeliveryQuote` as needed; `01-domain` has one version. Avoid `var`; values will cross validation, concurrency, and retry boundaries.

Exercise

1. Add a function that changes the recipient postcode of a request.
2. Add a function that changes the weight of parcel number `index`.
3. Write Kotest tests proving the input value was not changed.

Checkpoint

You can update nested structures with `copy`. Notice how much code the parcel update requires; keep that observation for the optics extra.

2. Validation: report everything wrong

Validate a request before a provider is called. Model individual problems, not prose strings.

```
enum class AddressField { SENDER, RECIPIENT }

sealed interface ShipmentValidationError {
    data object NoParcels : ShipmentValidationError
    data class InvalidPostcode(val field: AddressField, val value: String) :
    ↪  ShipmentValidationError
    data class InvalidWeight(val parcelIndex: Int, val weightGrams: Int) :
    ↪  ShipmentValidationError
    data class InvalidDimensions(val parcelIndex: Int) : ShipmentValidationError
}
```

A successful validation should *produce* something: a `Shipment` that only `validate` can create. Later code that takes a `Shipment` then knows the request is valid without checking again ("parse, don't validate").

```
@ConsistentCopyVisibility
data class Shipment internal constructor(val request: ShipmentRequest, val totalWeightGrams: Int)
```

A plain `ensure` inside `Raise` stops at the first failure. For independent checks use Arrow's accumulating builders, which run every branch and raise a `NonEmptyList` of all the failures:

- `zipOrAccumulate(a, b, c) { ... }` for a fixed number of independent checks;
- `mapOrAccumulate(items) { ... }` for every element of a collection;
- `withNel { ... }`, inside either one, to call a function that itself raises a `NonEmptyList`.

```
fun validate(request: ShipmentRequest): Either<NonEmptyList<ShipmentValidationError>, Shipment> =
    either {
        zipOrAccumulate(
            { validatePostcode(request.sender, AddressField.SENDER) },
            { validatePostcode(request.recipient, AddressField.RECIPIENT) },
            { withNel { validateParcels(request.parcels) } },
        ) { _, _, parcels -> Shipment(request, parcels.sumOf(Parcel::weightGrams)) }
    }

/** An invalid weight must not hide invalid dimensions on the same parcel. */
fun Raise<NonEmptyList<ShipmentValidationError>>.validateParcel(parcel: Parcel, index: Int):
↪  Parcel =
    zipOrAccumulate(
        { ensure(parcel.weightGrams in 1..30_000) { InvalidWeight(index, parcel.weightGrams) } },
        { ensure(parcel.dimensions.run { lengthMm > 0 && widthMm > 0 && heightMm > 0 }) {
        ↪  InvalidDimensions(index) } },
    ) { _, _ -> parcel }
```

These builders take a `Raise` *receiver*, so this section uses receiver style. Section 3 switches to context parameters.

Arrow 2.3 also has `accumulate { ensureOrAccumulate(...) }`, which reads more like a list of checks. It is still marked `@ExperimentalRaiseAccumulateApi`, so the reference solution sticks to the stable builders.

Exercise

1. Validate sender and recipient postcodes (4 digits in Norway and Denmark, `123 45` in Sweden), the parcel list, and every parcel.
2. Return all independent errors for a deliberately invalid request.
3. Add a test that asserts at least three distinct errors, comparing the error values themselves.

Checkpoint

Validation errors accumulate. That is useful because fixing one input field should not require submitting repeatedly to discover the next problem.

3. Typed domain errors

After validation, model failures that can happen while quoting a valid shipment.

```
sealed interface QuoteError {
    data class InvalidRequest(val problems: NonEmptyList<ShipmentValidationError>) : QuoteError
    data class UnsupportedRoute(val from: Country, val to: Country) : QuoteError
    data class ParcelTooHeavy(val totalWeightGrams: Int) : QuoteError
    data class NoServiceFor(val service: DeliveryService, val from: Country, val to: Country) :
    ↪  QuoteError
    data object NoQuotesAvailable : QuoteError
}

typealias QuoteShipmentError = QuoteError // Make it its own sealed type if the boundary needs
↪  more cases.
```

Write the business rules in a `Raise<QuoteError>` context. They should read as direct Kotlin: calculate, `ensure`, call, and return. Convert to `Either` only at the boundary.

```
import arrow.core.raise.context.ensure // the context-parameter versions of ensure, raise, bind,
↪  ...

context(_: Raise<QuoteError>)
fun checkQuotable(shipment: Shipment) {
    val from = shipment.request.sender.country
    val to = shipment.request.recipient.country
    ensure(from == to || (from to to) in crossBorderRoutes) { QuoteError.UnsupportedRoute(from,
↪  to) }
    ensure(shipment.totalWeightGrams <= 50_000) {
↪  QuoteError.ParcelTooHeavy(shipment.totalWeightGrams) }
    ensure(from == to || shipment.request.service != EXPRESS) { QuoteError.NoServiceFor(...) }
}

class QuoteService {
    fun quoteShipment(request: ShipmentRequest): Either<QuoteShipmentError, DeliveryQuote> =
        either {
            val shipment = validate(request).mapLeft(QuoteError::InvalidRequest).bind()
            quoteValidShipment(shipment)
        }

    context(_: Raise<QuoteError>)
    private fun quoteValidShipment(shipment: Shipment): DeliveryQuote {
        checkQuotable(shipment)
        raise(QuoteError.NoQuotesAvailable) // Section 5 replaces this with courier quotes.
    }
}
```

Context parameters must be named in Kotlin 2.4; `_` declares an anonymous one. The older context-receiver form `context(Raise<QuoteError>)` no longer compiles. Import `ensure` and `raise` from `arrow.core.raise.context` so they resolve against the context parameter. Inside `either { }`, the block's `Raise` receiver fills the context parameter automatically.

`mapLeft(QuoteError::InvalidRequest)` moves the accumulated input errors into the quote error type, rather than pretending they are provider failures.

Exercise

1. Add route and total-weight rules. Each parcel is at most 30 kg, but several valid parcels can still exceed the 50 kg shipment limit.
2. Make the public function return a typed failure for each rule.
3. Compare an `Either`-only implementation (`flatMap` chains) with the `Raise` implementation. Which is clearer inside a sequential use case?

Checkpoint

Explain why `NoQuotesAvailable` is not validation, and why it should usually short-circuit the quote flow. Then explain why these rules fail fast while validation accumulates.

4. Absence and existing Kotlin APIs

`insuranceAmount` and `pickupWindow` are genuinely optional. Start with nullable Kotlin types when absence is local and natural:

```
fun insuranceSurcharge(options: DeliveryOptions): Money =
    options.insuranceAmount?.let { Money(it.cents * 2 / 100) } ?: Money.ZERO
```

Introduce `Option` when it improves an API boundary or composition, for example when the value itself may legitimately be `null`. A cache of `Zone?` values needs to tell "not cached" (`None`) apart from "cached as no zone" (`Some(null)`), and a nullable type cannot. Do not wrap every nullable value automatically.

Use `Result` only when adapting an exception-oriented API, then classify the failure at the edge. Do not leak a raw `Result` through your business domain merely because an SDK returned one.

```
fun interface LegacyZoneLookup {
    fun lookup(postcode: String): Result<Zone> // fails with UnknownPostcodeException, or
    ↪ anything else
}

context(_: Raise<QuoteError>)
fun LegacyZoneLookup.zoneFor(address: Address): Zone =
    lookup(address.postcode).getOrElse { failure ->
        when (failure) {
            is UnknownPostcodeException -> raise(QuoteError.UnknownPostcode(address.postcode))
            else -> throw failure // technical: leave it to section 7
        }
    }
```

Exercise

1. Apply an insurance surcharge (2 %) only when insurance is present, and a pickup fee only when a pickup window is present.
2. Adapt the fake legacy postcode lookup returning `Result<Zone>`, and give `QuoteService` the lookup as a constructor parameter.
3. Decide whether unknown postcode is absence, a domain error, or a technical failure; write a test that documents your decision.

Checkpoint

`Option`, `Either`, `Result`, and nullable values express different things. Pick the smallest representation that says what the caller needs to know. The reference solution treats an unknown postcode as a domain error: the format already passed validation, so the request is well-formed and we simply do not deliver there.

5. Parallel courier quotes

Implement several fake courier clients. Their quote calls are independent suspend functions that either quote or decline.

```
sealed interface CourierDecline {
    val courier: Courier
    data class NoService(override val courier: Courier) : CourierDecline          // this courier
    ↪ can't
```

```
    data class Prohibited(override val courier: Courier, val reason: String) : CourierDecline //
    ↪ nobody may
}

interface CourierProvider {
    val courier: Courier
    suspend fun quote(shipment: Shipment): Either<CourierDecline, CourierQuote>
}
```

The exact error policy is part of the exercise:

- If one provider has no service, should it cancel all other providers?
- If every provider returns no service, which error should the caller see?
- When should independent provider failures be accumulated instead?

The reference policy: `NoService` concerns one courier, so skip it and keep the others running. `Prohibited` concerns the shipment itself, so raise `QuoteError.ShipmentProhibited`, which cancels every sibling still in flight. If nothing is left, raise `NoQuotesAvailable`. `Raise` is not a filter: skip with `null`, abort with `raise`.

```
context(_: Raise<QuoteError>)
suspend fun quoteOrSkip(provider: CourierProvider, shipment: Shipment): CourierQuote? =
    provider.quote(shipment).getOrElse { decline ->
        when (decline) {
            is CourierDecline.NoService -> null
            is CourierDecline.Prohibited -> raise(QuoteError.ShipmentProhibited(decline.courier,
            ↪ decline.reason))
        }
    }

context(_: Raise<QuoteError>)
suspend fun requestQuotes(shipment: Shipment, providers: List<CourierProvider>):
↪ List<CourierQuote> {
    val quotes = providers.parMapNotNull(concurrency = 3) { quoteOrSkip(it, shipment) }
    ensure(quotes.isNotEmpty()) { QuoteError.NoQuotesAvailable }
    return quotes
}
```

Use `parMap` or `parMapNotNull` with `concurrency` for a list of providers, and `parZip` for a fixed set of calls, which may have different result types. The concurrency happens inside the `Raise` scope, so a raised error cancels sibling work. Test cancellation with a fake that records `CancellationException`.

Dispatcher gotcha. In Arrow 2.3, `parMap` inherits the caller's coroutine context, but `parZip` and `raceN` *without* a context argument switch to `Dispatchers.Default`. Under `runTest` that means their delays run in real time and `currentTime` stays at 0. Pass `EmptyCoroutineContext` as the first argument to stay on the caller's dispatcher:

```
parZip(
    EmptyCoroutineContext,
    { quoteOrSkip(first, shipment) },
    { quoteOrSkip(second, shipment) },
) { a, b -> listOfNotNull(a, b) }
```

Finally, select the cheapest quote that meets the service's `maxDeliveryDays`, and add the surcharges from section 4. `QuoteService` now takes `providers` as well, and the quote flow is complete.

Exercise

1. Query three providers in parallel, and prove it with virtual time: three 1-second calls take 1 second.
2. Limit a larger provider list to a concurrency of three, and prove it by recording the peak number of calls in flight.
3. Return the cheapest eligible quote, with the others as alternatives.
4. Test that a fatal raised error cancels outstanding work.

Checkpoint

You have a useful vertical slice: validate a request, produce typed failures, fetch concurrent quotes, and return the selected quote.

6. Live quote updates

Add a `Flow<QuoteUpdate>` that emits each provider outcome as it arrives. Use Arrow's Flow operators to cap concurrent calls.

```
sealed interface QuoteUpdate {
    data class Received(val quote: CourierQuote) : QuoteUpdate
    data class Declined(val decline: CourierDecline) : QuoteUpdate
    data class Failed(val courier: Courier, val cause: Throwable) : QuoteUpdate
}
```

Decide whether consumer order matters:

- `parMap` preserves input ordering, so a slow first provider holds back the rest.
- `parMapUnordered` emits each result as soon as it completes.

Both Flow operators are marked `@FlowPreview` and `@ExperimentalCoroutinesApi`, so the functions that use them need `@OptIn`.

Keep the `Raise` scope out of a long-lived flow unless you understand its lifetime and cancellation behaviour. A practical design is to emit a typed result per provider and decide how to display or aggregate it in the flow consumer.

Turn exceptions into `Failed` with `Either.catch { }`, not `runCatching { }`. `runCatching` also catches `CancellationException`, so a cancelled provider would be reported as a failure and its coroutine would not stop. `Either.catch` (like Arrow's `catch`) rethrows it.

Exercise

1. Emit `QuoteUpdate.Received`, `QuoteUpdate.Declined`, and `QuoteUpdate.Failed` values.
2. Test that a slow first provider does not delay a later provider with unordered mapping.
3. Decide what should happen when the consumer cancels collection, for example with `first()`, and test that in-flight providers are cancelled.

7. Resilient provider integrations

Make fake providers intermittently fail with a technical exception. At the provider adapter boundary, decide whether the operation is safe to retry. Use decorators that implement `CourierProvider`, so the quote flow does not change.

Use Arrow resilience tools to:

- retry transient failures using a bounded `Schedule`;
- avoid retrying invalid requests, declined shipments, and non-idempotent booking operations;
- wrap an unhealthy provider in a circuit breaker;
- test retry and circuit state with controlled time.

```
class TransientProviderFailure(message: String) : RuntimeException(message)

// 100 ms, 200 ms, ... but at most two retries: `and` continues only while both do.
val schedule =
↪  Schedule.exponential<TransientProviderFailure>(100.milliseconds).and(Schedule.recurs(2))

class RetryingProvider(private val delegate: CourierProvider) : CourierProvider by delegate {
    override suspend fun quote(shipment: Shipment) = schedule.retry { delegate.quote(shipment) }
}

val breaker = CircuitBreaker(
    resetTimeout = 30.seconds,
    openingStrategy = CircuitBreaker.OpeningStrategy.Count(maxFailures = 3),
    timeSource = testTimeSource, // from runTest, so the reset timeout follows virtual time
)
```

`retry` with a `Schedule<TransientProviderFailure, *>` retries only that exception type and rethrows anything else immediately. A decline is a `Left` *value*, a successful answer, so it is never retried, and `protectOrThrow` does not count it as a failure. `OpeningStrategy.Count(n)` opens when the failure count *exceeds* `n`.

Order matters when composing: put the retry *inside* the breaker, so one logical quote counts as at most one failure and an open circuit is never retried. Then classify whatever technical failure is left at the edge as a new `CourierDecline.Unavailable(courier, cause)`, which the quote flow skips like `NoService`. One broken courier then cannot fail the whole quote.

Quoting is read-only, so it is safe to repeat. A booking call that charges money is not; only retry it if the courier API is idempotent, for example with an idempotency key.

Exercise

1. Make one provider fail twice and then return a quote; verify it is retried, and check the backoff with `currentTime`.
2. Verify a domain decline does not retry.
3. Open the circuit after the configured failures and verify further calls fail fast with `ExecutionRejected`, without calling the provider. Advance time past the reset timeout and verify a successful call closes it again.
4. Verify a quote still succeeds when one of two couriers is down.

Checkpoint

Retries and circuit breakers address technical availability. They do not make an unsupported route valid.

8. Resource safety

Add a provider client that needs acquisition and release, such as a simulated session or rate-limit permit.

```
fun SessionFactory.session(): Resource<ProviderSession> =
    resource(acquire = { open() }, release = { session, _ -> session.close() })

suspend fun <A> SessionFactory.withSession(block: suspend (ProviderSession) -> A): A =
    resourceScope { block(session().bind()) }
```

Use `resourceScope` so cleanup runs when work:

- succeeds;
- raises a typed error;
- throws an exception;
- is cancelled by a parallel sibling or the caller.

The release function receives an `ExitCase` (`Completed`, `Cancelled`, or `Failure`) if cleanup should differ. A typed `raise` reaches it as `Cancelled`, because `Raise` short-circuits with a cancellation exception.

Exercise

1. Record acquisition and release in a fake client.
2. Test release after success, a raised error, and an exception.
3. Test release after cancellation, both by the caller (`job.cancel()`) and by a sibling provider raising `Prohibited` in `requestQuotes`.

Checkpoint

Resource safety belongs with structured concurrency. `try/finally` remains valid Kotlin; `resourceScope` makes acquisition and release composition explicit, and runs release even while the coroutine is being cancelled.

Optional extras

These extensions are separate from the main learning route. Start them after the core workshop works. Reference solutions are in `09-extras`.

Optics

Return to the nested parcel update from section 1. Add Arrow Optics only after you have experienced the repetitive `copy` chain it replaces.

Since Arrow 2.3 the recommended setup is the optics **compiler plugin** rather than KSP. Add `alias(libs.plugins.arrow.optics)` (plugin id `io.arrow-kt.optics`) to `plugins { }` and `implementation(libs.arrow.optics)`, then annotate the classes with `@optics`. Unlike the KSP processor, the compiler plugin does not need an empty `companion object`. The plugin generates a lens per property on the companion:

```
ShipmentRequest.recipient.postcode.set(request, "7010")
ShipmentRequest.parcels.index(1).weightGrams.set(request, 250)  // import arrow.optics.dsl.index
ShipmentRequest.parcels.every.weightGrams.modify(request) { it + 10 }
```

Exercise: define an optic that focuses a parcel's weight and use it to update a nested request. Compare readability, discoverability, and debugging with the direct `copy` version. What happens with an index outside the list?

Racing

Add two redundant tracking providers. Query both and use Arrow racing to return the first *successful* tracking response. Define the difference between a fast failure and a successful result before writing code.

`raceN` returns whichever side *completes* first and cancels the other, so a fast failure would win. To race for success, a side that fails must not complete: it waits with `awaitCancellation()`, unless it is the last to fail, in which case it raises. Remember the dispatcher gotcha from section 5 and pass `EmptyCoroutineContext`.

Exercise: test that the losing request is cancelled and that a fast failed provider does not prevent a slower successful result when that is the chosen policy.

Transactional memory

Simulate a limited number of available courier pickup slots. Use STM to coordinate concurrent reservations safely. STM lives in a separate module: add `implementation(libs.arrow.fx.stm)`. Keep the slot count in a `TVar<Int>` and do the check and the decrement in one `atomically { }` block.

Exercise: launch many reservations concurrently on `Dispatchers.Default`, so they genuinely contend, and prove that the available slot count never becomes negative and exactly the capacity is reserved.

Sagas

Extend the workflow from quoting to booking: reserve courier capacity, charge the shipment, and arrange collection. Compensate completed steps when a later step fails. Each `saga({ action }) { compensation }` step registers its compensation; `transact()` runs the steps and, on failure, runs the completed steps' compensations in reverse order and then rethrows.

Exercise: simulate a pickup-arrangement failure after payment and verify the reservation and charge compensations run in the intended order.

Testing guide

Use Kotest tests as executable requirements. Prefer fakes over mocks for this workshop.

- Keep fake provider delays and outcomes configurable. `FakeCourier` in `05-parallel-quotes` takes a latency and a response lambda, and records calls and cancellation.
- Use `runTest` from `kotlinx-coroutines-test` for virtual time. `delay` returns instantly, and `currentTime` proves what ran in parallel. `currentTime` needs `@OptIn(ExperimentalCoroutinesApi::class)`.
- Assert errors structurally with `shouldBeLeft() shouldBe ...`, not through `toString()`.
- Test cancellation and cleanup explicitly.
- Keep one test per important policy decision: error classification, quote selection, retryability, and circuit behaviour.

Useful test names describe the business rule:

```
test("returns every invalid field in one response") {
    validate(request).shouldBeLeft() shouldContainExactlyInAnyOrder listOf(/* ... */)
}
```

```
test("does not retry a courier that declines the route") {
    // ...
}

test("releases a provider session when a sibling quote fails") {
    // ...
}
```

The reference stages also run kotlinter and detekt on `./gradlew check`. The starter does not, so lint never gets in the way of an exercise.

Further reading

- Arrow typed errors
- Arrow concurrency and resources
- Arrow resilience
- Arrow immutable data
- Kotlin context parameters
- Kotlin structured concurrency
- Testing coroutines

Questions to take away

- Which errors should the caller be able to handle, and how does the type make that clear?
- Which parallel calls should cancel one another, and which should complete independently?
- What makes a failed operation safe to retry?
- Where does direct immutable Kotlin remain clearest, and where would optics earn their complexity?